

1.3.10:

(2)

(1)

(0)

// Precedence (x) / > (+; -) > ("(",
 // gặp toán hạng → đưa vào c² pg
 // gặp (: đưa vào stack
 // gặp) : rút cái toán tử ra khỏi stack từ chỉ
 // chừa "("

// gặp toán tử :

// Nếu toán tử cuối Stack có ưu tiên $\geq$ current,
 rút nó đưa vào c² pg
 // Đưa dư thừa / chỉ toán tử ở dưới có ưu tiên
 thấp hơn, hoặc empty, hoặc "("
 Solution 1.3.10

Stack (String) stack = new Stack();
 StringBuilder sb = new StringBuilder();
 while (input.hasNext())

token = input.next();

if (token != "(") : stack.push("(");
 else if (token == "+" || "-" || "*" || "/" || "%") :

while (!stack.isEmpty() && stack.peek().precedence < token.precedence)

sb.append(token);

if (stack.peek() == "(")

stack.pop();

else if (token == "=")

stack.pop(); sb.append(token);

while (!stack.isEmpty() && stack.peek() == "(")

stack.pop();

sb.append(stack.pop());

stack.pop();

else : sb.append(token);

while (!stack.isEmpty()) sb.append(stack.pop());
 sout(sb);

1.3.11: $\rightarrow$ có một đ. và mđ

Evaluate Post Fix

```
Stack<String> operator = new Stack<>();
Stack<Integer> val = new Stack<>();
while (input.hasNext()) {
    current = input.next();
    if (current == "+" || "-" || "*" || "/") {
```

```
        operator push(current);
    } else if (current == "+") {
```

```
        int val2 = val.pop();
```

```
        int val1 = val.pop();
```

```
        op = val.operator.pop();
        switch (op) {
```

```
            case "+": val.push(val1 + val2);
```

```
            case "-": val.push(val1 - val2);
```

```
            case "*": val.push(val1 * val2);
```

```
            case "/": val.push(val1 / val2);
```

```
        } else { val.push(current);
```

```
        if (!stack.isEmpty())
```

```
            sout(stack.pop());
```

1.3.12

```
public class StackCopyClient
```

```
public static Stack<String> copy(Stack<String> original stack)
```

```
{
    Stack<String> temp = new Stack<>();
```

```
    Stack<String> res = new Stack<>();
```

```
    for (String s : original)
```

```
        temp.push(s);
```

```
    while (temp.isEmpty())
```

```
        res.push(temp.pop());
```

```
    return res;
```

1.3.13 Only a la output char

1.3.14 :
class ResizingArrayQueueOfStr {

String[] q;
int head;

int tail;

int n;

public ResizingArrayQueueOfStr() {
q = new String[2];

head = 0;

tail = 0;

n = 0;

}

public void resize (int capacity)

String[] copy = new String[capacity];

for (int i = 0; i < n; i++)

copy[i] = q[(head + i) % q.length];

q = copy;

head = 0;

tail = 0;

public void enqueue (String item)

if (n == q.length) {

resize(2 * q.length)

q[tail] = item;

tail++;

if (tail >= q.length) {

tail = 0;

n++;



```
public String dequeue() {
    if (empty) throw E(" - ")
    String item = q[head]
    if (head == null)
        head++;
}
```

```
if (head == q.length)
    head = 0
```

```
n--;
if (n > 0 && n == q.length / 4)
    resize(q.length / 2);
return item;
```

1.3.15.

// A B C D E, k = 3 → out: C

but we don't know, user enter how many times

→ ~~Don't~~ Can't determine a specific variable N
 → So we use sliding window, always keep ~~q~~ queue.size() = k. When finished, output is queue.~~pop~~ dequeue().

Reusable Reusing class 1.3.14

```
while (q input.hasNext()) {
    queue.add(input.next());
}
```

```
if (queue.size() > k)
    queue.poll().dequeue();
return queue.dequeue();
```

1.3.1
 isFull() {
 return N == capacity;
 }

1.3.3 : chẵn b, f) → Sai
 Ví dụ : 023567 → Sai
~~1.3.4~~

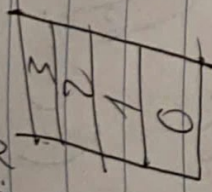
1.3.4
 public boolean Parentheses (Str input) {
 Stack<Character> stack = new Stack();
 char[] ch = input.toCharArray();
 for (char s: ch) {
 if (s == "(" or s == "[" or s == "{")
 stack.push(s);
 } else {
 if (stack.push() == "(" and s == ")") rtf;
 if (!stack.push() == "[" and s == "]") rtf;
 if (!stack.push() == "{" and s == "}") rtf;
 stack.pop();
 }
 }
 return stack.isEmpty();
 }

1.3.5 :
~~N = 50 : 50 % 2 = 0~~

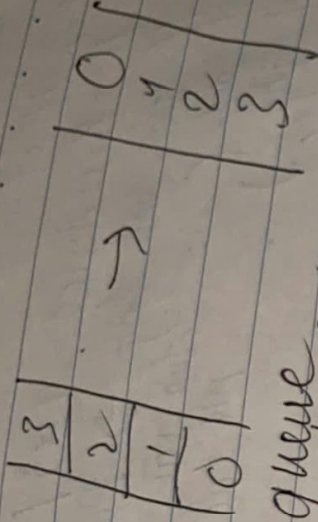
80 | 2
 0 | 25 | 2
 1 | 12 | 2
 0 | 6 | 2
 0 | 3 | 2
 1 | 1 | 2
 0

→ in ra 110010
 in biểu d' như phân của
 N (N = 50)

1.3.6



Stack



Queue

Đoạn code này xử lý các phần tử trong queue.

1.3.7

```
public item peek() {
```

```
    return top.data; } // C LINKED LIST
```

1.3.9

Solution 1.3.9

```
Stack<String> operator = new Stack<>();
Stack<String> operand = new Stack<>();
while (input.hasNext()) {
```

```
    String token = input.next();
```

```
    if (token == "+" || "-" || "*" || "/" || "%")
```

```
        operator.push(token);
```

```
    else if (token == "(")
```

```
        String substr = "(" + val1 + "op" + val2 + ")";
```

```
        val2 = operand.pop();
```

```
        val1 = operand.pop();
```

```
        op = operator.pop();
```

```
        operand.push(substr);
```

```
    } else {
```

```
        operand.push(token);
```

```
    }
    return operand.pop();
```